

# DEEP REINFORCEMENT LEARNING

## 365.382 (SS26)

Tabular Value-Based Methods



Sohvi Luukkonen

LIT AI LAB

Institute for Machine Learning

# Today's Outline

- Recap: What is RL?
- RL Taxonomy
- Bellman equations
- Planning by dynamic programming
  - Solve a *known* MDP
- Model-free prediction
  - Estimate the value function of an *unknown* MDP
- Model-free control
  - Optimize the value function of an *unknown* MDP

# Recap & Resources

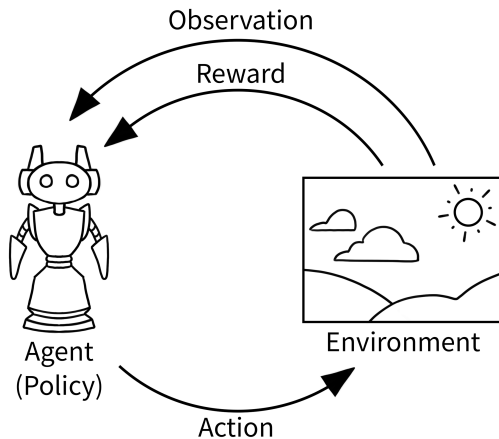
- This is a recap to review main/basic concepts
- Basic concepts should be clear after this recap.
- If not, you should invest some additional time to catch up
- [Reinforcement Learning: An Introduction](#) [Chapters 3 to 7, 12]
- [DeepMind x UCL | RL Lecture Series 2021](#) [Lectures 3 to 6]

# **What is Reinforcement Learning?**

# What is Reinforcement Learning?

Computational approach of **learning to make decisions** from **interactions**.

# The Interaction Loop



**Goal:** maximize cumulative reward (called the **return**)

# Markov Decision Process

## Definition (Markov Decision Process)

A *Markov Decision Process* is a tuple  $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \gamma \rangle$ , where

- $\mathcal{S}$  is the set of all **states**
- $\mathcal{A}$  is the set of all **actions**
- $\mathcal{P}$  are the **joint transition probabilities** with **Markov property**
  - $p(r, s' | s, a)$  is the joint transition probability of a reward  $r$  and next state  $s'$ , given state  $s$  and action  $a$
  - $p$  defines the **dynamics** of the problem
- $\gamma \in [0, 1]$  is a **discount factor** weighting near vs. far future rewards

# Markov Decision Process (Alternative Definition)

## Definition (Markov Decision Process)

A *Markov Decision Process* is a tuple  $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ , where

- $\mathcal{S}$  is the set of all **states**
- $\mathcal{A}$  is the set of all **actions**
- $\mathcal{P}$  are the **state transition probabilities** with **Markov property**
  - $p(s'|s, a) = \sum_r p(s', r|s, a)$  is the transition probability of next state  $s'$ , given state  $s$  and action  $a$
  - $p$  defines the **dynamics** of the problem
- $\mathcal{R}$  is a **reward** function
  - $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$  is the **expected reward**, achieved on a transition starting in  $(s, a)$ :  
$$r(s, a) = \mathbb{E}[R|s, a] = \sum_r r \sum_{s'} p(s', r|s, a)$$
- $\gamma \in [0, 1]$  is a **discount factor** weighting near vs. far future rewards

# Policies and Goal of an RL Agent

## Definition (Policy)

A **policy**  $\pi$  is a mapping  $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$  that assigns to each state–action pair  $(s, a)$  the **probability of selecting action  $a$  in state  $s$** . It is denoted by  $\pi(a|s)$ , with the constraint  $\sum_{a' \in \mathcal{A}} \pi(a'|s) = 1, \forall s \in \mathcal{S}$ .

## Definition (Goal of an RL Agent )

To find a behavior policy  $\pi$  that maximizes the (**expected**) return  $G_t$

# Discounted Returns

## Definition

The discounted **returns**  $G_t$  is the sum of discounted future rewards:

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) = R_{t+1} + \gamma G_{t+1} \end{aligned}$$

- $\gamma \in [0, 1)$  makes the sum finite
- $\gamma \rightarrow 0$  only the immediate reward will be considered
- $\gamma \rightarrow 1$  rewards far in the future are taken into account more strongly
- recursive formulation

# Value Functions

## Definition

The **state-value function**  $v_{\pi}(s)$  of an MDP is the expected return starting from the state  $s$ , and then following the policy  $\pi$ :

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s]$$

The **action-value function**  $q_{\pi}(s, a)$  of an MDP is the expected return starting from the state  $s$ , taking action  $a$ , and then following the policy  $\pi$ :

$$q_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a]$$

# Optimal Value Functions

## Definition

The **optimal state-value function**  $v^*(s)$  is the maximum state-value function over all policies:

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

The **optimal action-value function**  $q^*(s, a)$  is the maximum action-value function over all policies:

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

- The optimal value function gives the best possible performance in the MDP.
- The MDP is "solved" when the optimal value function is known.

# Finding an Optimal Policy

An optimal policy can be found by maximizing over  $q^*(s, a)$ ,

$$\pi^*(s, a) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_a q^*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

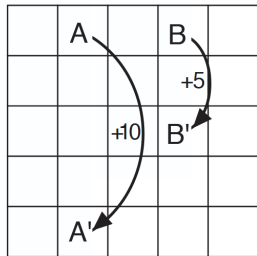
- There is always a **deterministic optimal policy** for any MDP
- If we **know**  $q^*(s, a)$ , we immediately have the **optimal policy**
- There can be **multiple** optimal policies
  - If multiple actions maximize  $q^*(s, \cdot)$ , we can (stochastically) pick any of these

# Prediction and Control

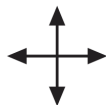
- **Prediction** or **policy evaluation**: estimating  $v_\pi$  or  $q_\pi$ 
  - Given a policy, what is the expected return under that behavior?
- **Control** or **policy optimization**: estimating  $v^*$  or  $q^*$ 
  - What is the optimal way of behaving? What is the optimal value function?

# Gridworld Example – Prediction

What is the value function for the uniform random policy?



(a)



Actions

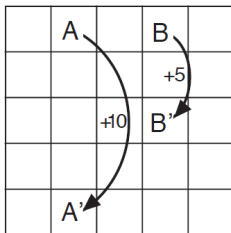
|      |      |      |      |      |
|------|------|------|------|------|
| 3.3  | 8.8  | 4.4  | 5.3  | 1.5  |
| 1.5  | 3.0  | 2.3  | 1.9  | 0.5  |
| 0.1  | 0.7  | 0.7  | 0.4  | -0.4 |
| -1.0 | -0.4 | -0.4 | -0.6 | -1.2 |
| -1.9 | -1.3 | -1.2 | -1.4 | -2.0 |

(b)

Reward is  $-1$  when bumping into the wall,  $\gamma = 0.9$ .

# Gridworld Example – Control

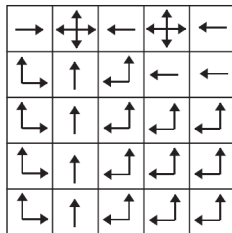
What is the optimal value function over all possible policies? What is the optimal policy?



a) gridworld

|      |      |      |      |      |
|------|------|------|------|------|
| 22.0 | 24.4 | 22.0 | 19.4 | 17.5 |
| 19.8 | 22.0 | 19.8 | 17.8 | 16.0 |
| 17.8 | 19.8 | 17.8 | 16.0 | 14.4 |
| 16.0 | 17.8 | 16.0 | 14.4 | 13.0 |
| 14.4 | 16.0 | 14.4 | 13.0 | 11.7 |

b)  $V^*$



c)  $\pi^*$

Reward is  $-1$  when bumping into the wall,  $\gamma = 0.9$ .

# RL Taxonomy

# RL Taxonomy 1

## ■ Value-based

- ☐ Learned value function
- ☐ Implicit policy ( $\epsilon$ -greedy)

## ■ Policy-based

- ☐ Learned policy
- ☐ No value function

## ■ Actor-Critic

- ☐ Learned policy (actor)
- ☐ Learned value function (critic)

# RL Taxonomy 2

## ■ On-policy RL

- ☐ Learn  $\pi$  with samples from  $\pi$

## ■ Off-policy RL

- ☐ Learn  $\pi$  with samples from  $\mu$

## ■ Offline RL

- ☐ Learn  $\pi$  from pre-collected samples from (unknown)  $\mu$

# RL Taxonomy 3

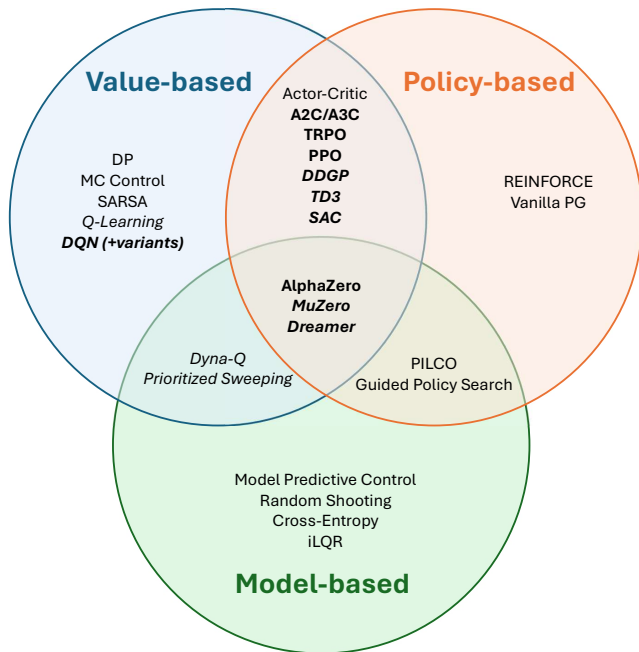
## ■ Model-free

- ☐ Policy and/or value function
- ☐ No model

## ■ Model-based

- ☐ Policy and/or value function
- ☐ Model

# RL Taxonomy



# Bellman Equations

# Value Function

The value function  $v(s)$  gives the long-term value of the state  $s$ , and it can be defined recursively:

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}_{\pi} [G_t | S_t = s] = \mathbb{E} [G_t | S_t = s, \pi] \\&= \mathbb{E} [R_{t+1} + \gamma G_{t+1} | S_t = s, \pi] \\&= \mathbb{E} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s, A_t \sim \pi(S_t)] \\&= \sum_a \pi(a|s) \sum_r \sum_{s'} p(s', r | s, a) [r + \gamma v_{\pi}(s')] \\&= \sum_a \pi(a|s) \left[ r(s, a) + \gamma \sum_{s'} p(s' | s, a) v_{\pi}(s') \right]\end{aligned}$$

# Action-Value Function

The value function  $q(s, a)$  gives the long-term value of the state-action  $(s, a)$  pair, and it can be defined recursively:

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_{\pi} [R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\ &= \mathbb{E}_{\pi} [R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \\ &= \sum_r \sum_{s'} p(s', r \mid s, a) \left[ r + \gamma \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a') \right] \\ &= r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a') \end{aligned}$$

# Bellman Expectation Equations

## Theorem (Bellman Expectation Equations)

Given an MDP,  $\langle S, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ , for *any policy*  $\pi$ , the *value functions* obey the following expectation equations:

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}_{\pi} [r + \gamma v_{\pi}(s') | S = s] \\&= \sum_a \pi(a|s) \left[ r(s, a) + \gamma \sum_{s'} p(s'|s, a) v_{\pi}(s') \right] \\q_{\pi}(s, a) &= \mathbb{E}_{\pi} [r + \gamma q_{\pi}(s', a') | S = s, A = a] \\&= r(s, a) + \gamma \sum_{s'} p(s'|s, a) \sum_{a'} \pi(a'|s') q_{\pi}(s', a')\end{aligned}$$

The Bellman equation *averages* over *all possible outcomes*, *weighting each* by its *probability*.

# Bellman Optimality Equations

## Theorem (Bellman Optimality Equations)

Given an MDP,  $\langle S, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ , the *optimal value functions* obey the following expectation equations:

$$v^*(s) = \max_a \left[ r(s, a) + \gamma \sum_{s'} p(s'|s, a) v^*(s') \right]$$
$$q^*(s, a) = r(s, a) + \gamma \sum_{s'} p(s'|a, s) \max_{a'} q^*(s', a')$$

# Some Intuition

Greedy on  $v^*$  = Optimal policy

An optimal policy can be found by maximizing over  $q^*(s, a)$ ,

$$\pi^*(s, a) = \begin{cases} 1 & \text{if } a = \operatorname{argmax}_a q^*(s, a) \\ 0 & \text{otherwise} \end{cases}$$

Apply the Bellman Expectation Eq.

$$q_{\pi^*}(s, a) = r(s, a) + \gamma \sum_{s'} p(s'|s, a) \underbrace{\sum_{a'} \pi^*(a'|s') q_{\pi^*}(s', a')}_{\max_{a'} q^*(s', a')}$$

# Solving the Bellman Equations

## Prediction

- Expectation eqs. are linear
- Closed-form solution, but complexity is  $O(|\mathcal{S}|^3)$
- Iterative methods for larger problems
  - ☐ Using model: DP
  - ☐ Using samples: MC, TD

## Control

- Optimality eqs. are non-linear
- No closed-form solution
- Iterative methods
  - ☐ Using model: DP
  - ☐ Using samples: MC, SARSA, Q-Learning

# **Planning with Dynamic Programming**

# What is Dynamic Programming?

*Dynamic programming refers to a collection of algorithms that can be used to compute optimal policies **given a perfect model** of the environment as a Markov decision process (MDP).*

All methods consist of two important parts:

- **policy evaluation**
- **policy improvement**

# Policy Evaluation

Value-function:

$$v_{\pi}(s) = \mathbb{E}_{\pi} [r + \gamma v_{\pi}(s') | S_t = s]$$

We can turn the equality into an update algorithm

## Algorithm: Iterative Policy Evaluation

- 1: Input: Policy  $\pi$  to be evaluated
- 2: Initialize  $V_0 = C$
- 3: **repeat**
- 4:    $\forall s : V_{k+1}(s) \leftarrow \mathbb{E}_{\pi} [r + \gamma V_k(s') | S_t = s]$
- 5: **until**  $\max_{s \in \mathcal{S}} |V_k(s) - V_{k-1}(s)| < \epsilon$
- 6: **return**  $V_k \approx v_{\pi}$

# Policy Improvement

How can we improve the policy  $\pi$ ?

## Theorem (Policy Improvement Theorem)

*Let  $\pi$  and  $\pi'$  be any pair of deterministic policies s.t.,  $\forall s \in \mathcal{S}, q_\pi(s, \pi'(s)) \geq v_\pi(s)$ . Then policy  $\pi'$  must be as good as or better than  $\pi$ :  $v_{\pi'}(s) \geq v_\pi(s), \quad \forall s \in \mathcal{S}$ .*

Natural choice: the **greedy policy** (always satisfies the improvement condition)

## Greedy Policy Improvement

$$\pi'(s) = \operatorname{argmax}_a q_\pi(s, a) = \operatorname{argmax}_a \mathbb{E}_\pi \left[ r + \gamma v_\pi(s') \right]$$

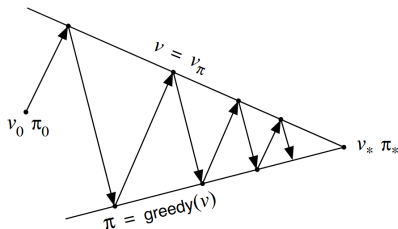
- If  $v_{\pi'}(s) = v_\pi(s)$  for all  $s$ , then both policies are **optimal**

# Policy Iteration

## Algorithm: Policy Iteration

- 1: Initialize  $V_0$  and  $\pi_0$  arbitrarily
- 2: **repeat**
- 3:     Policy Evaluation:  $V_{\pi_i} \leftarrow$  evaluate  $\pi_i$  using iterative policy evaluation
- 4:     Policy Improvement:  $\pi_{i+1} \leftarrow$  greedy policy w.r.t.  $V_{\pi_i}$
- 5: **until**  $\pi_i = \pi_{i-1}$
- 6: **return**  $\pi_i$  (optimal policy),  $V_{\pi_i}$  (optimal value function)

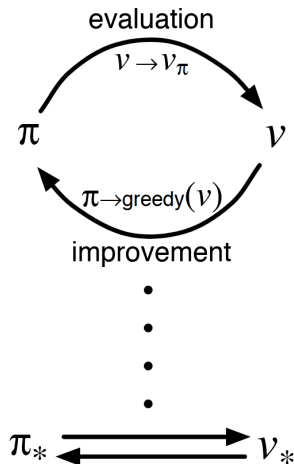
# Policy Iteration



**Policy evaluation:** estimate  $v_\pi$

**Policy improvement:** generate  $\pi' \geq \pi$  (e.g., greedy)

- Policy evaluation is run to convergence
- Converges to the optimal policy
- Each iteration is computationally expensive



# Value Iteration

Does policy evaluation need to be run to convergence?

No, we can interleave policy evaluation and improvement steps.

(Reminder) Bellman optimality equation:

$$v^*(s) = \max_a \left[ r(s, a) + \gamma \sum_{s'} p(s'|s, a) v^*(s') \right]$$

we can turn the equality into an update algorithm.

## Algorithm: Value Iteration

- 1: Initialize  $V_0$  arbitrarily
- 2: **repeat**
- 3:    $\forall s : V_{k+1}(s) \leftarrow \max_a \mathbb{E}_\pi [r + \gamma V_k(s') | S_t = s]$
- 4: **until**  $\max_{s \in \mathcal{S}} |V_k(s) - V_{k-1}(s)| < \epsilon$
- 5: **return**  $\pi_k$  (optimal policy),  $V_k$  (optimal value function)

# No Model?

- DP methods require a perfect model of the environment
- In many cases, we do not have access to such a model
- We need to estimate the expectations with samples from the environment

# Model-Free Prediction

# Monte Carlo Algorithms

- Sample experiences to learn without a model
- Monte Carlo methods learn from complete episodes
- MC is a model-free method for policy evaluation

# Monte Carlo Prediction

Goal: learn  $v_\pi$  from episodes generated by  $\pi$ :  $S_0, A_0, R_1, \dots, S_T \sim \pi$ . The **return** is the total discounted reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$$

The value function is the expected return:

$$v_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

for which we can use the **sample average** return estimate instead of the expectation.

# Monte Carlo Update Rule

(Reminder) Incrementally Approximating Value Functions:

$$New \leftarrow Old + Stepsize[Target - Old]$$

Monte Carlo update rule:

$$V_{\text{new}}(S_t) \leftarrow V_{\text{old}}(S_t) + \alpha_t[G_t - V_{\text{old}}(S_t)]$$

- $G_t$  is an unbiased estimate of  $v_\pi(S_t)$
- Convergence to  $v_\pi$  if  $\alpha_t$  is decayed appropriately
- But when episodes are long, the variance of  $G_t$  can be very high and learning can be very slow

# Temporal Difference Learning

(Reminder) Bellman equation:

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s]$$

(Reminder) We can approximate iteratively (policy evaluation in DP):

$$V_{\text{new}}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma V_{\text{old}}(S_{t+1}) | S_t = s]$$

We can **sample** this to **estimate returns** without waiting for the episode to end:

$$V_{t+1}(S_t) = R_{t+1} + \gamma V_t(S_{t+1})$$

## TD(0) Update Rule

$$V_{\text{new}}(S_t) \leftarrow V_{\text{old}}(S_t) + \alpha \left[ \underbrace{R_{t+1} + \gamma V_{\text{old}}(S_{t+1})}_{\text{target}} - V_{\text{old}}(S_t) \right]$$

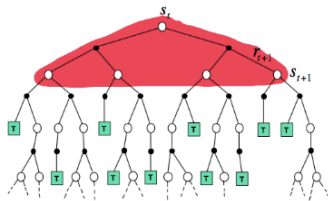
TD error =  $\delta_t$

- Similarly to MC, TD(0) is a **model-free** method
- Similarly to DP, TD(0) is a **bootstrapping** method (uses estimates of returns)
- TD(0) can learn **online** after every step, without waiting for the episode to end

# Policy Evaluation Algorithms

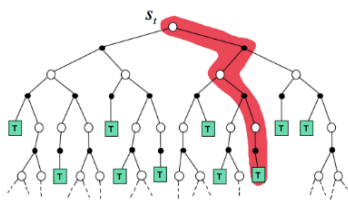
## Dynamic Programming

$$V(S_t) \leftarrow \mathbb{E}_{\pi} [R_{t+1} + \gamma V(S_{t+1})]$$



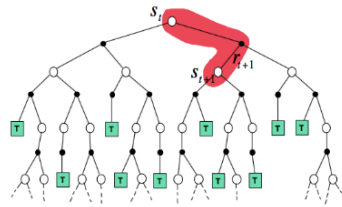
## Monte-Carlo

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$



## Temporal-Difference

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$



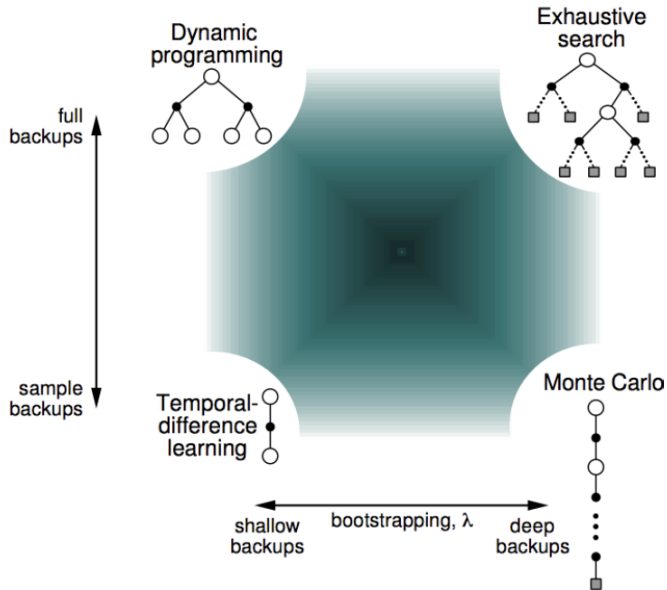
## Bootstrapping

update involves an estimate  
(DP & TD)

## Sampling

update samples an expectation  
(MC & TD)

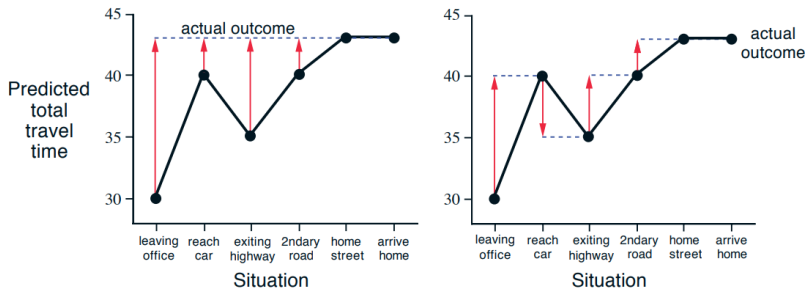
# Unified View of RL



## MC vs. TD (Core Differences)

- **TD**: online learning with single-step updates; no final outcome needed
  - **TD**: works with incomplete sequences and continuing environments
  - **MC**: updates only after episode end when the full return is known
  - **MC**: requires complete episodes and storing trajectories
- 
- **TD** uses value estimates as targets, which can be inaccurate early in learning

# MC vs. TD (Bias, Variance, and Propagation)

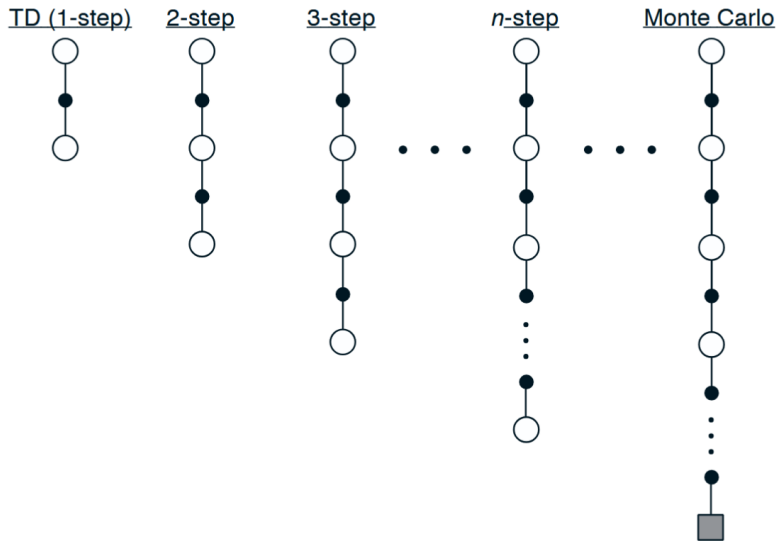


**Figure 6.1:** Changes recommended in the driving home example by Monte Carlo methods (left) and TD methods (right).

- **MC** return is an unbiased estimate of  $v_\pi$ , but updates are noisier
- **TD** target is biased, but often has lower variance and can learn faster
- In **TD**, information can propagate back through states relatively slowly
- In **MC**, information propagates faster along full trajectories

# Multi-Step Bootstrapping

# Multi-Step Prediction



# Multi-Step Returns

$$n = 1 \quad (TD) \quad G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$$

$$n = 2 \quad G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$$

$$\vdots \quad \quad \quad \vdots$$

$$n = \infty \quad (MC) \quad G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T$$

$n$ -step return:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

Multi-step TD update:

$$V_{\text{new}}(S_t) \leftarrow V_{\text{old}}(S_t) + \alpha \left[ G_t^{(n)} - V_{\text{old}}(S_t) \right]$$

## Mixed Multi-Step Returns: $\lambda$ -return

Multi-step returns bootstrap once, at  $S_{t+n}$ :

$$G_t^{(n)} = R_{t+1} + \gamma G_{t+1}^{(n-1)} \quad n > 1$$

$$G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1}) \quad \text{1-step bootstrap}$$

The  $\lambda$ -return averages over many  $n$ -step returns:

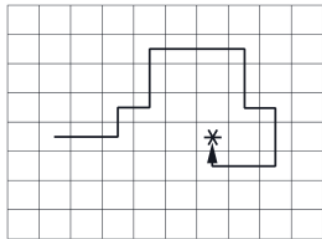
$$\begin{aligned} G_t^\lambda &= (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)} + \lambda^{T-t-1} G_t \\ &= R_{t+1} + \gamma \left[ (1 - \lambda) V(S_{t+1}) + \lambda G_{t+1}^\lambda \right] \end{aligned}$$

Special cases:

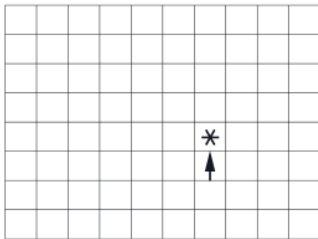
- $\lambda = 0$ : TD return
- $\lambda = 1$ : Monte Carlo return

# TD( $\lambda$ ): Temporal Difference with Eligibility traces

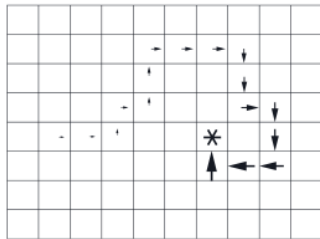
Path taken



Action values increased  
by one-step TD



Action values increased  
by TD ( $\lambda$ ) with  $\lambda=0.9$



Benefits from both TD and MC:

- TD-like: low variance, online learning, bootstrapping
- MC-like: faster propagation of information, less bias

# Model-Free Control

# Model-Free Policy Iteration Using Action-Value Function

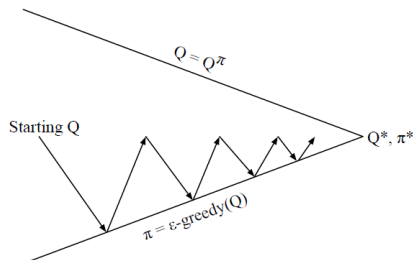
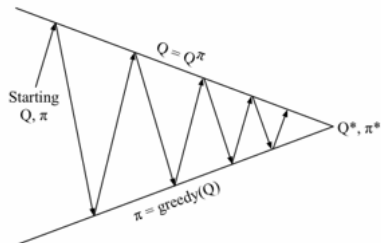
Greedy policy improvement over  $v(s)$  requires model of MDP dynamics:

$$\pi'(s) = \operatorname{argmax}_a \mathbb{E} \left[ R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s, A_t = a \right]$$

Greedy policy improvement over  $q(s, a)$  is **model-free**:

$$\pi'(s) = \operatorname{argmax}_a q_{\pi}(s, a)$$

# Generalized Policy Iteration with Action-Value Function



- **Policy evaluation:** MC or TD policy evaluation,  $Q \approx q_\pi$
- **Policy improvement:** greedy policy improvement,  $\pi' = \text{greedy}(Q)$
- **Problem:** No exploration (won't sample all action-state pairs)
- **Solution:**  $\epsilon$ -greedy policy improvement

# On-Policy Model-Free Control

MC Update rule:

$$Q_{\text{new}}(S_t, A_t) \leftarrow Q_{\text{old}}(S_t, A_t) + \alpha \left[ G_t - Q_{\text{old}}(S_t, A_t) \right]$$

TD Update rule (SARSA):

$$Q_{\text{new}}(S_t, A_t) \leftarrow Q_{\text{old}}(S_t, A_t) + \alpha \left[ R_{t+1} + \gamma Q_{\text{old}}(S_{t+1}, A_{t+1}) - Q_{\text{old}}(S_t, A_t) \right]$$

# Monte Carlo Control

## Monte Carlo Control Algorithm

- 1: **repeat**
- 2:     Generate an episode  $S_0, A_0, R_1, \dots, S_T$  following  $\epsilon$ -greedy policy  $\pi$
- 3:     **for** each state-action pair  $(S_t, A_t)$  appearing in the episode **do**
- 4:          $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t [G_t - Q(S_t, A_t)]$
- 5:         (with  $\alpha_t$  decreasing, e.g.  $1/N(S_t, A_t)$ )
- 6:     **end for**
- 7:     Update  $\pi$  to be  $\epsilon$ -greedy w.r.t.  $Q$
- 8: **until** convergence
- 9: **return**  $\pi$  (optimal policy),  $q$  (optimal action-value function)

# TD Control: SARSA

## SARSA Algorithm

```
1: repeat  
2:   Initialize  $S_0$   
3:   Choose  $A_0$  from  $S_0$  using  $\epsilon$ -greedy policy derived from  $Q$   
4:   while  $S_t$  is not terminal do  
5:     Take action  $A_t$ , observe  $R_{t+1}$  and  $S_{t+1}$   
6:     Choose  $A_{t+1}$  from  $S_{t+1}$  using  $\epsilon$ -greedy policy derived from  $Q$   
7:      $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$   
8:      $S_t \leftarrow S_{t+1}, A_t \leftarrow A_{t+1}$   
9:   end while  
10: until convergence  
11: return  $\pi$  (optimal policy),  $q$  (optimal action-value function)
```

# GLIE

## Definition (Greedy in the Limit with Infinite Exploration)

A policy is GLIE if it satisfies the following conditions:

- **Infinite exploration**: Every state-action pair is visited infinitely often.
- **Greedy in the limit**: The policy converges to a greedy policy with respect to the action-value function.

For example,  $\epsilon$ -greedy policies with  $\epsilon_t \rightarrow 0$  and  $\sum_{t=1}^{\infty} \epsilon_t = \infty$  are GLIE.

## Theorem

*GLIE Model-free control converges to the optimal action-value function  $q^*$  and the optimal policy  $\pi^*$ .*

# On and Off-Policy Learning

- **On-policy learning:** Learn about **behavior** policy  $\pi$  from experiences generated by  $\pi$
- **Off-policy learning:** Learn about **target** policy  $\pi$  from experiences generated by a different **behavior** policy  $\mu$ 
  - ☐ Learn from observing humans or other agents (e.g., from logged data)
  - ☐ Re-use experience from old policies (e.g., from your own past experience)
  - ☐ Learn about multiple policies while following one policy
  - ☐ Learn about **greedy** policy while following **exploratory** policy

# Importance Sampling

- Learn about  $\pi$  from data generated by  $\mu$  (behavior policy)
- **Reweight samples** so they look as if they were generated by  $\pi$

$$\mathbb{E}_{\pi}[G_t] = \mathbb{E}_{\mu} \left[ \frac{\pi(A_t|S_t)}{\mu(A_t|S_t)} G_t \right]$$

- The ratio corrects for the probability difference between the two policies:

$$\rho_t = \frac{\pi(A_t|S_t)}{\mu(A_t|S_t)}$$

- For full trajectories:

$$\rho_t^T = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{\mu(A_k|S_k)}$$

- Intuition:
  - ☐ trajectories likely under  $\pi$  get higher weight
  - ☐ trajectories unlikely under  $\pi$  get lower weight

# Off-Policy Model Free Control with Importance Sampling

Off-Policy Monte Carlo Update rule:

$$Q_{\text{new}}(S_t, A_t) \leftarrow Q_{\text{old}}(S_t, A_t) + \alpha \left[ \rho_t G_t^T - Q_{\text{old}}(S_t, A_t) \right]$$

Off-Policy TD Update rule (Off-Policy SARSA):

$$Q_{\text{new}}(S_t, A_t) \leftarrow Q_{\text{old}}(S_t, A_t) + \alpha \left[ \rho_t (R_{t+1} + \gamma Q_{\text{old}}(S_{t+1}, A_{t+1})) - Q_{\text{old}}(S_t, A_t) \right]$$

# Off-Policy TD Control: Q-Learning

(Reminder) Bellman optimality equation for  $q^*$ :

$$q^*(s, a) = \mathbb{E} \left[ R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \mid S_t = s, A_t = a \right]$$

Q-Learning update rule:

$$Q_{\text{new}}(S_t, A_t) \leftarrow Q_{\text{old}}(S_t, A_t) + \alpha \left[ R_{t+1} + \gamma \max_{a'} Q_{\text{old}}(S_{t+1}, a') - Q_{\text{old}}(S_t, A_t) \right]$$

is a sample-based approximation of the Bellman optimality equation that estimates the value of the **greedy** policy.

# Summary

- **Bellman equations** characterize the value functions of any policy and the optimal value functions
- **Dynamic programming** methods can solve the Bellman equations given a perfect model of the environment
- **Model-free prediction** methods can learn value functions from experience without a model
- **Model-free control** methods can learn optimal policies from experience without a model

Next week:

- More on Q-Learning
- Function approximation (deep RL)